

進階語法(3)

- 本章將介紹如何使用增強型枚舉來管理控制台顏色，以及如何使用擴展為現有類型添加新功能
- 這些功能是利用 Dart 的高階特性，提升命令列應用程式的使用者體驗，從而使應用程式更具互動性和視覺吸引力。

使用枚舉與延展來擴張應用程式

- 準備工作：
 - 將 dartex4 專案目錄，複製一份成 dartex5，做為開發專案資料夾。

```
cp -r dartex4 dartex5
```

- 複製之前，請注意目前所在的工作目錄。
- Task 目標：
 - 透過為輸出添加顏色和改進文字格式，改善 Dartpedia CLI 應用程式的使用者體驗。
- Task1：增強控制台顏色枚舉
 - 作法：利用 ConsoleColor 枚舉類型，為控制台輸出添加顏色，其類型包含 RGB 值和用於為文字套用顏色的方法。
 - 建立並開啟 command_runner/lib/src/console.dart 檔案。
 - 在該檔案內，新增以下程式碼以定義 ConsoleColor 枚舉：

```
import 'dart:io';

const String ansiEscapeLiteral = '\x1B';

/// 使用 `\\n` 字元分割字串，然後將每行內容輸出到控制台。
/// 由 duration 變數定義每行輸出之間的時間間隔（以毫秒為單位）。

Future<void> write(String text, {int duration = 50}) async {

  final List<String> lines = text.split('\\n');

  for (final String l in lines) {
    await _delayedPrint('$l \\n', duration: duration);
  }
}

/// 一行接一行列出

Future<void> _delayedPrint(String text, {int duration = 0}) async {

  return Future<void>.delayed(
    Duration(milliseconds: duration),
    () => stdout.write(text),
  );
}
```

[Back to top](#)

```
/// RGB 格式顏色是用於設定輸入框樣式
/// 所有顏色均來自「 Dart 品牌風格指南 」

/// 作為演示版本，僅包含此程式關心的顏色。
/// 如果想要使用更多顏色，請在這裡添加。

enum ConsoleColor {

  /// 天空藍 - #b8eafe
  lightBlue(184, 234, 254),

  /// Dart 品牌指南中的強調色
  /// 警告紅 - #F25D50

  red(242, 93, 80),

  /// 輕黃色 - #F9F8C4

  yellow(249, 248, 196),

  /// 輕灰色，好用於文字 - #F8F9FA

  grey(240, 240, 240),

  /// 白色

  white(255, 255, 255);

  const ConsoleColor(this.r, this.g, this.b);

  final int r;
  final int g;
  final int b;
}
```

- 此枚舉定義一組控制台顏色及其對應的 RGB 值。每種顏色都是 ConsoleColor 枚舉的常數實例。
- 在 ConsoleColor 枚舉中新增用於為文字套用顏色的方法：

```
// 開啟 command_runner/lib/src/console.dart
enum ConsoleColor {

  // (中間省略....)

  const ConsoleColor(this.r, this.g, this.b);

  final int r;
  final int g;
  final int b;

  /// 更改所有未來輸出的文字顏色（直到重新設定）
```

```

/// ```dart
/// print('hello'); // 以終端預設顏色列印
/// print(ConsoleColor.red.enableForeground);
/// print('hello'); // 印出紅色內容
/// ```

String get enableForeground => '$ansiEscapeLiteral[38;2;$r;$g;${b}m';

/// 更改所有未來輸出的文字顏色（直到重新設定）
/// ```dart
/// print('hello'); // 以終端預設顏色列印
/// print(ConsoleColor.red.enableBackground);
/// print('hello'); // 印出紅色背景色
/// ```

String get enableBackground => '$ansiEscapeLiteral[48;2;$r;$g;${b}m';

/// 將文字和背景顏色重設為終端預設值

static String get reset => '$ansiEscapeLiteral[0m';

/// 設定輸入框的文字顏色

String applyForeground(String text) {
  return '$ansiEscapeLiteral[38;2;$r;$g;${b}m$text$reset';
}

/// 設定背景顏色，然後重設顏色更改
String applyBackground(String text) {
  return '$ansiEscapeLiteral[48;2;$r;$g;${b}m$text$ansiEscapeLiteral[0m';
}
}

```

- 這些方法使用 ANSI 轉義碼為文字套用前景色和背景色。
 - applyForeground 和 applyBackground 方法傳回一個應用 ANSI 轉義碼的字串。
- Task2: 建立字串延展
 - 接下來，在 String 類別上建立一個延展，並增加用於控制台顏色和格式化文字的實際方法。
 - 將以下程式碼加入 command_runner/lib/src/console.dart 檔案：

```

extension TextRenderUtils on String {
  String get errorText => ConsoleColor.red.applyForeground(this);
  String get instructionText => ConsoleColor.yellow.applyForeground(this);
  String get titleText => ConsoleColor.lightBlue.applyForeground(this);

  List<String> splitLinesByLength(int length) {

    final List<String> words = split(' ');
    final List<String> output = <String>[];

    final StringBuffer strBuffer = StringBuffer();

    for (int i = 0; i < words.length; i++) {

```

```

        final String word = words[i];

        if (strBuffer.length + word.length <= length) {
            strBuffer.write(word.trim());

            if (strBuffer.length + 1 <= length) {
                strBuffer.write(' ');
            }
        }

        // 如果下一個單字長度超過限制，則換行。

        if (i + 1 < words.length &&
            words[i + 1].length + strBuffer.length + 1 > length) {
            output.add(strBuffer.toString().trim());
            strBuffer.clear();
        }
    }

    // 加入剩餘內容
    output.add(strBuffer.toString().trim());

    return output;
}
}

```

- 上述程式碼定義一個名為 `TextRenderUtils` 的 `String` 類別延展。
 - 此處添加三個用於控制台顏色的 getter 方法：`errorText`、`instructionText` 和 `titleText`。
 - 另外，還新增一個名為 `splitLinesByLength` 的方法，用於將字串分割成指定長度。
- Task3: 更新 `command_runner` 軟體封包
 - 開啟 `command_runner/lib/command_runner.dart` 檔案，並新增以下程式碼：

```

library;

export 'src/arguments.dart';
export 'src/command_runner_base.dart';
export 'src/exceptions.dart';
export 'src/help_command.dart';
export 'src/console.dart'; // 增加此行

// TODO: 匯出所有供此軟體封包用戶端使用的函式庫。

```

- Task4: 執行彩色輸出指令
 - 最後，實作一個範例指令來測試列印功能。
 - 對於會使用 Dart 套件的開發者而言，實作套件的範例用法是一個很好的練習。
 - 這個範例建立一個指令，可以使控制台輸出彩色。
 - 開啟 `command_runner/example/command_runner_example.dart` 檔案
 - 將檔案內容替換為以下程式碼：

```
import 'dart:async';

import 'package:command_runner/command_runner.dart';

class PrettyEcho extends Command {
  PrettyEcho() {
    addFlag(
      'blue-only',
      abbr: 'b',
      help: 'When true, the echoed text will all be blue.',
    );
  }

  @override
  String get name => 'echo';

  @override
  bool get requiresArgument => true;

  @override
  String get description => 'Print input, but colorful.';

  @override
  String? get help =>
    'echos a String provided as an argument with ANSI coloring,';

  @override
  String? get valueHelp => 'STRING';

  @override
  FutureOr<String> run(ArgResults arg) {
    if (arg.commandArg == null) {
      throw ArgumentError(
        'This argument requires one positional argument',
        name,
      );
    }

    List<String> prettyWords = [];
    var words = arg.commandArg!.split(' ');
    for (var i = 0; i < words.length; i++) {
      var word = words[i];
      switch (i % 3) {
        case 0:
          prettyWords.add(word.titleText);
        case 1:
          prettyWords.add(word.instructionText);
        case 2:
          prettyWords.add(word.errorText);
      }
    }
  }
}
```

```
        return prettyWords.join(' ');
    }
}

void main(List<String> arguments) {
    final runner = CommandRunner()..addCommand(PrettyEcho());

    runner.run(arguments);
}
```

- 上述程式碼定義一個繼承自 Command 類別的 PrettyEcho 指令。
 - 它接受一個字串作為參數，並根據每個單字在字串中的位置為其應用不同的顏色。
 - run 方法使用 TextRenderUtils 擴充功能中的 titleText、instructionText 和 errorText getter 方法來套用顏色。
- 切換至 dartex5/command_runner 目錄內，並且執行下列指令：

```
dart run example/command_runner_example.dart echo "Hello World ThankYou"
```

- 執行結果應該如下：

```
Hello World ThankYou
```

參考文獻

- [Dart 官方教學文件](#)

Generated by [aglio](#) on 13 May 2026